Largest Island

AlgoExpert · Hard · Graphs

The problem

- Matrix of 0 (land) and 1 (water).
- Flip at most one water cell to land to maximise the largest 4-connected land island; return its size.
- Label islands with ids and sizes first.

Intuition

- Label each island with its size, then for each water cell sum the distinct neighbouring islands plus one.

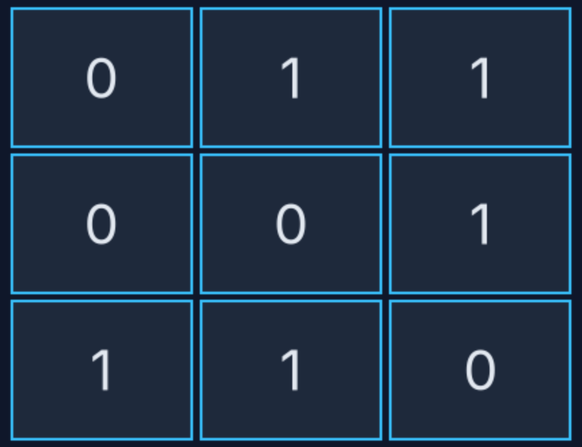
Brute force vs optimal

- Flip each water cell and recount → $O((w \cdot h)^2)$
- Label islands once, then check each water cell's distinct neighbours → $O(w \cdot h)$
- Use a set so an island isn't counted twice

Approach

- 1. Flood-fill to label islands and record sizes
- 2. For each water cell, collect unique neighbouring labels
- 3. Sum sizes plus one
- 4. Take the max

Worked example



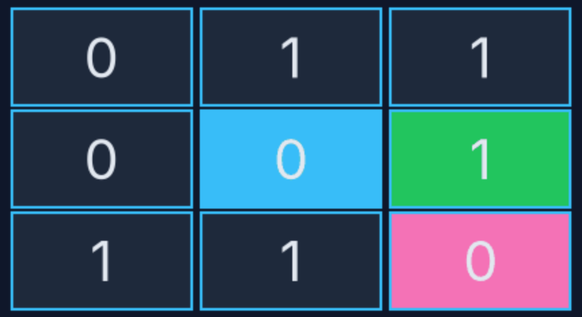
0 = land, 1 = water: flip one 1 to maximise the island

Worked example



Label islands: A = 3, B = 1

Worked example




```
flip (1,2): touches A (3) + B (1) → 1 + 3 + 1 = 5
```

Flip (1,2): 1 + A + B = 5

Java solution (1/2)

```java
public class LargestIsland {
    public static int largestIsland(int[][] matrix) {
        int R = matrix.length, C = matrix[0].length, id = 2;
        int[][] m = new int[R][];
        for (int r = 0; r < R; r++) m[r] = matrix[r].clone();
        Map<Integer, Integer> size = new HashMap<>();
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int best = 0;
        for (int r = 0; r < R; r++) for (int c = 0; c < C; c++) {
            if (m[r][c] != 0) continue;
            int s = 0;
            ArrayDeque<int[]> st = new ArrayDeque<>();
            st.push(new int[]{r, c});
            m[r][c] = id;
            while (!st.isEmpty()) {
                int[] p = st.pop(); s++;
                for (int[] d : dirs) {
                    int nr = p[0] + d[0], nc = p[1] + d[1];
                    if (nr >= 0 && nc >= 0 && nr < R && nc < C && m[nr][nc] == 0) { m[nr][nc] = id; st.push(new int[]{nr, nc}); }
                }
            }
            size.put(id++, s);
```

Java solution (2/2)

```
            best = Math.max(best, s);
        }
        for (int r = 0; r < R; r++) for (int c = 0; c < C; c++) {
            if (m[r][c] != 1) continue;
            Set<Integer> ids = new HashSet<>();
            for (int[] d : dirs) {
                int nr = r + d[0], nc = c + d[1];
                if (nr >= 0 && nc >= 0 && nr < R && nc < C && m[nr][nc] > 1) ids.add(m[nr][nc]);
            }
            int s = 1;
            for (int i : ids) s += size.get(i);
            best = Math.max(best, s);
        }
        return best;
    }
}
```

Complexity

- Time: O(w*h)
- Space: O(w*h)

Edge cases & pitfalls

- Count each neighbouring island once (set of IDs)
- No water → whole grid
- No land → 1

Interview tips & follow-ups

- Same as LeetCode Making a Large Island

Recap

- Label each island with its size, then for each water cell sum the distinct neighbouring islands plus one.
- Complexity: O(w*h)